

LAB 9

NAME: Hajra Sarwar

REG NO: 2023-BSE-022

CLASS: BSE-5A

SUBJECT: CC

LAB TITLE: Codespaces & AWS: GH CLI (Codespaces), AWS CLI, EC2, IAM, Security Groups, Filters & Queries

TASK:

Task 1: GitHub CLI, Codespace setup and authentication

Goal: Install GH CLI, authenticate, create/connect Codespace.

Steps:

1. Install GitHub CLI (Windows example):

```
HP@DESKTOP-JFEH5TK MINGW64 /d/Github CLI
$ winget install --id GitHub.cli
Found GitHub CLI [GitHub.cli] Version 2.83.2
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/cli/cli/releases/download/v2.83.2/gh_2.83.2_windows_amd64.msi
17.7 MB / 17.7 MB
Successfully verified installer hash
Starting package install...
Successfully installed
```

2. Authenticate GH CLI for Codespaces:

```
PS C:\Users\HP> gh auth login -s codespace
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 7482-952F
Press Enter to open https://github.com/login/device in your browser...
! Authentication complete.
- gh config set -h github.com git_protocol https
! Configured git protocol
! Logged in as hajrasarwar11
PS C:\Users\HP>
```

Optional: List available Codespaces:

```
PS C:\Users\HP> gh codespace list
```

NAME	DISPLAY NAME	REPOSITORY	BRANCH	STATE	CREATED AT
psychic-tribble-wrq756xxww9x2964g	psychic tribble	hajrasarwar11/Skill-Bridge	main*	Shutdown	about 12 days ago
fluffy-tribble-wrq756x47vrX25q6q	fluffy tribble	hajrasarwar11/CC-HajraSarwar-022	main	Shutdown	about 4 hours ago

```
PS C:\Users\HP>
```

4. Create or connect to Codespace:

```
PS C:\Users\HP> gh codespace ssh -c fluffy-tribble-wrq756x47vrX25q6q
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

codespaces-5ad799:/workspaces/CC-HajraSarwar-022$
```

Task 2: Install AWS CLI inside the Codespace and configure it

Goal: Install and configure AWS CLI.

Steps:

1. Download and install AWS CLI:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ curl -L "https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.15.25.zip" -o awscli.zip
% Total    % Received % Xferd  Average Speed   Time    Time     Time
Dload  Upload   Total    Spent    Left  Speed
100 57.4M 100 57.4M    0     0 232M      0 --:--:-- --:--:-- --:--:-- 233M
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ unzip awscli.zip
Archive:  awscli.zip
replace aws/README.md? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
(46/49) Installing aws-cli (2.27.25-r0)
(47/49) Installing aws-cli-pyc (2.27.25-r0)
(48/49) Installing aws-cli-bash-completion (2.27.25-r0)
(49/49) Installing aws-cli-zsh-completion (2.27.25-r0)
Executing busybox-1.37.0-r20.trigger
OK: 620 MiB in 258 packages
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws --version
aws-cli/2.27.25 Python/3.12.12 Linux/6.8.0-1030-azure source/x86_64.alpine.3
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$
```

2. Configure AWS CLI:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws configure
AWS Access Key ID [*****faam]: AKIA5WPGPNRITZBBKOL
AWS Secret Access Key [*****faam]: do1utgxV0vW65PJ87N4n370E4DGldfQIwx1wo3d9
Default region name [me-central-1]: me-central-1
Default output format [json]: json
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$
```

3. Verify connectivity:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ cat ~/.aws/credentials
[default]
aws_access_key_id = AKIA5WPGPNRITZBBKOL
aws_secret_access_key = do1utgxV0vW65PJ87N4n370E4DGldfQIwx1wo3d9
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ cat ~/.aws/config
[default]
region = me-central-1
output = json
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws sts get-caller-identity
{
  "UserId": "AIDA5WPGPNRISWOC332W5",
  "Account": "941618064465",
  "Arn": "arn:aws:iam:941618064465:user/Admin"
}
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$
```

Task 3: Create security group and add ingress rules using Codespace IP

Goal: Create EC2 security group, add SSH & HTTP rules from Codespace IP.

Steps:

1. Create security group (replace VPC id):

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws ec2 create-security-group --group-name 'MySecurityGroup' \
> --description 'My Security Group' \
> --vpc-id 'vpc-05883252ecb09ac3e'
{
  "GroupId": "sg-0388043fbd4cc7348",
  "SecurityGroupArn": "arn:aws:ec2:me-central-1:941618064465:security-group/sg-0388043fbd4cc7348"
}
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$
```

2. Inspect security group:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-security-groups --group-ids sg-0388043fbd4cc7348
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0388043fbd4cc7348",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-05883252ecb09ac3e",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:941618064465:security-group/sg-0388043fbd4cc7348",
      "OwnerId": "941618064465",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": []
    }
  ]
}
```

3. Get Codespace public IP:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ curl icanhazip.com
4.240.18.226
```

4. Authorize SSH inbound on port 22:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws ec2 authorize-security-group-ingress \
> --group-id sg-0388043fbd4cc7348 \
> --protocol tcp \
> --port 22 \
> --cidr 4.240.18.226/32
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-07c7d219e6cd0b10c",
      "GroupId": "sg-0388043fbd4cc7348",
      "GroupOwnerId": "941618064465",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "4.240.18.226/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:941618064465:security-group-rule/sgr-07c7d219e6cd0b10c"
    }
  ]
}
```

5. Verify SSH ingress rule:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-security-groups --group-ids sg-0388043fbd4cc7348
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0388043fbd4cc7348",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-05883252ecb09ac3e",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:941618064465:security-group/sg-0388043fbd4cc7348",
      "OwnerId": "941618064465",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 22,
          "ToPort": 22,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.18.226/32"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ]
    }
  ]
}
```

6. Add HTTP rule on port 80:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws ec2 authorize-security-group-ingress \
> --group-id 'sg-0388043fbd4cc7348' \
> --ip-permissions '{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"4.240.18.226/32"}]}'
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-024ca47dcec7ae273",
      "GroupId": "sg-0388043fbd4cc7348",
      "GroupOwnerId": "941618064465",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "4.240.18.226/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:941618064465:security-group-rule/sgr-024ca47dcec7ae273"
    }
  ]
}
```

Verify both ingress rules:

```
codespaces-5ad799:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-security-groups --group-ids sg-0388043fbd4cc7348
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0388043fbd4cc7348",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-05883252ecb09ac3e",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:941618064465:security-group/sg-0388043fbd4cc7348",
      "OwnerId": "941618064465",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 22,
          "ToPort": 22,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.18.226/32"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        },
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.18.226/32"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ]
    }
  ]
}
```

Task 4: Create key pair, launch EC2 instance, SSH

Goal: Create ED25519 key, launch EC2 instance, test SSH connectivity.

Steps:

1. Create key pair and save PEM:

```
codespaces-3c16f6:~$ cd /tmp
codespaces-3c16f6:/tmp$
codespaces-3c16f6:/tmp$ aws ec2 create-key-pair \
> --key-name MyED25519Key \
> --key-type ed25519 \
> --key-format pem \
> --query 'KeyMaterial' \
> --output text > MyED25519Key.pem
MyED25519Key.pem
codespaces-3c16f6:/tmp$
codespaces-3c16f6:/tmp$ chmod 400 MyED25519Key.pem
codespaces-3c16f6:/tmp$ ls -l MyED25519Key.pem
-r----- 1 vscode vscode 388 Jan 18 04:47 MyED25519Key.pem
codespaces-3c16f6:/tmp$
```

2. View created key pairs:

```
codespaces-3c16f6:/tmp$ aws ec2 describe-key-pairs
{
  "KeyPairs": [
    {
      "KeyPairId": "key-0c8ea4a47ad1f7275",
      "KeyType": "rsa",
      "Tags": [],
      "CreateTime": "2025-12-30T18:32:38.069000+00:00",
      "KeyName": "assignment2-key",
      "KeyFingerprint": "b0:f5:6a:52:74:84:39:b7:38:72:18:91:93:8f:17:83:0c:21:ba:20"
    },
    {
      "KeyPairId": "key-0139ac453bedc25e9",
      "KeyType": "ed25519",
      "Tags": [],
      "CreateTime": "2026-01-18T04:47:05.341000+00:00",
      "KeyName": "MyED25519Key",
      "KeyFingerprint": "e87YVtwq8FQNFtUft1hj2+Eo2ge05het0P244RJ2EyY="
    }
  ]
}
```

3. (Optional) Delete key pair:

```
codespaces-3c16f6:/tmp$ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyPairId": "key-0139ac453bedc25e9"
}
```

4. Launch EC2 instance:

```
codespaces-3c16f6:/workspaces/CC-Hajrasarwar-022$ aws ec2 run-instances \
> --image-id ami-05e6edf2ba1fcb7dea \
> --count 1 \
> --instance-type t3.micro \
> --key-name MyED25519Key \
> --security-group-ids sg-0388043fbd4cc7348 \
> --subnet-id subnet-015aa793c89df0a82 \
> --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
{
  "ReservationId": "r-0d6c4cb9450c6afe2",
  "OwnerId": "941618064465",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "d0fa29ac-5e0b-4551-bd99-cd06a207be5a",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2026-01-18T05:22:08+00:00",
            "AttachmentId": "eni-attach-0a1587957d31ec13f",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          },
          "Description": "",
          "Groups": [
            {
              "GroupId": "sg-0388043fbd4cc7348",
              "GroupName": "MySecurityGroup"
            }
          ],
          "Ipv6Addresses": [],
          "MacAddress": "06:e6:c9:c3:67:55",
          "NetworkInterfaceId": "eni-06de4abb8eb854ea9",
          "OwnerId": "941618064465",
          "PrivateDnsName": "ip-172-31-44-181.me-central-1.compute.internal",
          "PrivateIpAddress": "172.31.44.181",
          "PrivateAddresses": [

```

5. Get public IP:

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
> --output table
```

DescribeInstances	
i-0351e10b8e498bbf0	3.29.93.0

- SSH into instance (fix permission if needed):

[illegible]

7. Stop, start, (optional) terminate instance:

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 stop-instances --instance-ids i-0351e10b8e498bbf0
{
  "StoppingInstances": [
    {
      "InstanceId": "i-0351e10b8e498bbf0",
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances --instance-ids i-0351e10b8e498bbf0 --query "Reservations[*].Instances[*].State.Name" --output table
DescribeInstances
+-----+
| InstanceId |
+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 start-instances --instance-ids i-0351e10b8e498bbf0
{
  "StartingInstances": [
    {
      "InstanceId": "i-0351e10b8e498bbf0",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances --instance-ids i-0351e10b8e498bbf0 --query "Reservations[*].Instances[*].State.Name" --output table
DescribeInstances
+-----+
| InstanceId |
+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 terminate-instances --instance-ids i-0351e10b8e498bbf0
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0351e10b8e498bbf0",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

Task 5: Understand AWS describe-* commands

1. Describe security groups

```
codespaces-3c16fe:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-security-groups
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0388043fbd4cc7348",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "1"
        }
      ]
    }
  ]
}
```

2. Describe VPCs

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-vpcs
{
  "Vpcs": [
    {
      "OwnerId": "941618064465",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-0c2c5f1357e516f26",
          "CidrBlock": "172.31.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": true,
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-05883252ecb09ac3e",
      "State": "available",
      "CidrBlock": "172.31.0.0/16",
      "DhcpOptionsId": "dopt-077fe63bbc5909d99"
    }
  ]
}
```

3. Describe subnets

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-subnets
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az1",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "941618064465",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "SubnetArn": "arn:aws:ec2:me-central-1:941618064465:subnet/subnet-015aa793c89df0a82"
    }
  ]
}
```

4. Describe instances

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances
{
  "Reservations": [
    {
      "ReservationId": "r-0d6c4cb9450c6afe2",
      "OwnerId": "941618064465",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [],
          "ClientToken": "d0fa29ac-5e0b-4551-bd99-cd06a207be5a",
          "EbsOptimized": false
        }
      ]
    }
  ]
}
```

5. Describe regions

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-regions
{
  "Regions": [
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-south-1",
      "Endpoint": "ec2.ap-south-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-north-1"
    }
  ]
}
```

6. Describe availability zones

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-availability-zones
{
  "AvailabilityZones": [
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1a",
      "ZoneId": "mec1-az1",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1"
    }
  ]
}
```

Task 6: IAM: create group, user, attach policies, login & keys

1–16: IAM steps with commands (as described in previous message)

1. Create Group

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam create-group --group-name MyGroupCli
{
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPA5WPGPNRIWZ2DXGI5O",
    "Arn": "arn:aws:iam:941618064465:group/MyGroupCli",
    "CreateDate": "2026-01-18T06:10:22+00:00"
  }
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```


2. Add Group Details

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam get-group --group-name MyGroupCli
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPA5WPGPNRIWZ2DXGI50",
    "Arn": "arn:aws:iam::941618064465:group/MyGroupCli",
    "CreateDate": "2026-01-18T06:10:22+00:00"
  }
}
```

3. Create User

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam create-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDA5WPGPNRI5CHMKDSEZ",
    "Arn": "arn:aws:iam::941618064465:user/MyUserCli",
    "CreateDate": "2026-01-18T06:13:27+00:00"
  }
}
```

4. Add User to Group and see Details

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam add-user-to-group --user-name MyUserCli --group-name MyGroupCli
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam get-group --group-name MyGroupCli
{
  "Users": [
    {
      "Path": "/",
      "UserName": "MyUserCli",
      "UserId": "AIDA5WPGPNRI5CHMKDSEZ",
      "Arn": "arn:aws:iam::941618064465:user/MyUserCli",
      "CreateDate": "2026-01-18T06:13:27+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPA5WPGPNRIWZ2DXGI50",
    "Arn": "arn:aws:iam::941618064465:group/MyGroupCli",
    "CreateDate": "2026-01-18T06:10:22+00:00"
  }
}
```

5. List policies:

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-policies \
> --query "Policies[?contains(PolicyName, 'EC2')].(Name:PolicyName)" \
> --output text
AmazonEC2FullAccess
AmazonEC2ReadOnlyAccess
AmazonElasticMapReduceforEC2Role
AmazonEC2RoleforDataPipelineRole
AmazonEC2ContainerServiceforEC2Role
AmazonEC2ContainerServiceRole
```

6. Create login profile and sign:

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam create-login-profile \
> --user-name MyUserCli \
> --password 111 \
> --password-reset-required

An error occurred (PasswordPolicyViolation) when calling the CreateLoginProfile operation: Password should have a minimum length of 8
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam create-login-profile --user-name MyUserCli --password hlofaam13579 --password-reset-required

An error occurred (PasswordPolicyViolation) when calling the CreateLoginProfile operation: Password should meet 1 more of the following requirements: Password should have at least one upper
case letter, Password should have at least one symbol
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam create-login-profile --user-name MyUserCli --password Hlofaam13579 --password-reset-required
{
  "LoginProfile": {
    "UserName": "MyUserCli",
    "CreateDate": "2026-01-18T06:24:31+00:00",
    "PasswordResetRequired": true
  }
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam attach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
```

7. Create Access Key:

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIA5WPGPNRIT6KGBDFN",
      "Status": "Active",
      "CreateDate": "2026-01-18T06:25:40+00:00"
    }
  ]
}
```

Task 7: Filters: query with filters to find instances

1. Filter by Name tag → Public IP

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --filters "Name=tag:Name,Values=MyServer" \
> --query "Reservations[*].Instances[*].PublicIpAddress" \
> --output text
158.252.35.126
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```

2. Filter by instance type

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --filters "Name=instance-type,Values=t3.micro" \
> --query "Reservations[.Instances[.InstanceId]" \
> --output table
-----
| DescribeInstances |
+-----+
| i-0351e10b8e498bbf0 |
| i-0a2fff9c29c487b2b |
+-----+
```

3. Filter by subnet

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --filters "Name=subnet-id,Values=subnet-0600df5fa8ce60857" \
> --query "Reservations[*].Instances[*].InstanceId" \
> --output table
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```

4. Filter by VPC

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --filters "Name=vpc-id,Values=vpc-06be85cd81b657192" \
> --query "Reservations[*].Instances[*].InstanceId" \
> --output table
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```

Task 8: Use --query to format outputs for reporting

1. Table: InstanceId, PublicIpAddress, Name tag

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --filters "Name=tag:Name,Values=MyServer" \
> --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
> --output table
-----
| InstanceId | PublicIpAddress | Name |
+-----+
| i-0351e10b8e498bbf0 | 158.252.35.126 | MyServer |
| i-0a2fff9c29c487b2b | 158.252.35.126 | MyServer |
+-----+
```

2. Table: InstanceId, state

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
> --output table
-----
| InstanceId | State |
+-----+
| i-0351e10b8e498bbf0 | terminated |
| i-0a2fff9c29c487b2b | running |
+-----+
```

3. Table: InstanceId, InstanceType, Availability Zone

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances \
> --query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
> --output table
-----
| InstanceId | InstanceType | AvailabilityZone |
+-----+
| i-0351e10b8e498bbf0 | t3.micro | me-central-1a |
| i-0a2fff9c29c487b2b | t3.micro | me-central-1a |
+-----+
```


Cleanup: Remove all resources

1. Terminate instances

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 terminate-instances --instance-ids i-0a2fff9c29c487b2b
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0a2fff9c29c487b2b",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

2. Delete EBS volumes & snapshots

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-volumes --query "Volumes[*].[VolumeId,State,Attachments]" --output table
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```

3. Delete security group & key pair

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-security-groups --query "SecurityGroups[*].[GroupId,GroupName,VpcId]" --output table
-----
DescribeSecurityGroups
-----
+-----+-----+-----+
| sg-0388043fbd4cc7348 | mysecuritygroup | vpc-05883252ecb09ac3e |
| sg-0567ae2c936d1bb1c | default         | vpc-05883252ecb09ac3e |
+-----+-----+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 delete-security-group --group-id sg-0388043fbd4cc7348
{
  "Return": true,
  "GroupId": "sg-0388043fbd4cc7348"
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-key-pairs --query "KeyPairs[*].[KeyName,KeyFingerprint]" --output table
-----
DescribeKeyPairs
-----
+-----+-----+
| assignment2-key | b01f5:6a:52:74:84:29:b7:28:72:18:91:93:17:83:0c:21:ba:20 |
| MyED25519Key    | LFxmdgr37A0QK+ADCEy3TKGZ/Xo9ftCcB2ozqFYs450+         |
+-----+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyPairId": "key-05cbd3a8807fd4750"
}
```

4. Delete IAM users, keys, groups

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIA5WPGPNRIT6KGBDFN",
      "Status": "Active",
      "CreateDate": "2026-01-18T06:25:40+00:00"
    }
  ]
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam delete-access-key --user-name MyUserCli --access-key-id AKIA5WPGPNRIT6KGBDFN
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": []
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam delete-login-profile --user-name MyUserCli
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-groups-for-user --user-name MyUserCli
{
  "Groups": [
    {
      "Path": "/",
      "GroupName": "MyGroupCli",
      "GroupId": "AGP5WPGPNRIMZ2DXG150",
      "Arn": "arn:aws:iam::941618064465:group/MyGroupCli",
      "CreateDate": "2026-01-18T06:10:22+00:00"
    }
  ]
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam remove-user-from-group --user-name MyUserCli --group-name MyGroupCli
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam delete-user --user-name MyUserCli
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-attached-group-policies --group-name MyGroupCli
{
  "AttachedPolicies": [
    {
      "PolicyName": "AWSEC2SqlHaInstancePolicy",
      "PolicyArn": "arn:aws:iam::aws:policy/AWSEC2SqlHaInstancePolicy"
    }
  ]
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam detach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName
An error occurred (NoSuchEntity) when calling the DetachGroupPolicy operation: Policy arn:aws:iam::aws:policy/EXAMPLEPolicyName was not found.
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam detach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/AWSEC2SqlHaInstancePolicy
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam delete-group --group-name MyGroupCli
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```

5. Final verification (billing/resources)

```
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-instances --query "Reservations[*].Instances[*].InstanceId,State.Name" --output table
-----+-----+-----+
| DescribeInstances |
+-----+-----+-----+
| i-0a2ff9c20c487b2b | terminated |
+-----+-----+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-volumes --query "Volumes[*].VolumeId,State" --output table
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-key-pairs --query "KeyPairs[*].KeyName" --output table
-----+-----+-----+
| DescribeKeyPairs |
+-----+-----+-----+
| assignment2-key |
+-----+-----+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws ec2 describe-security-groups --query "SecurityGroups[*].GroupName" --output table
|DescribeSecurityGroups|
+-----+-----+-----+
| default |
+-----+-----+-----+
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Admin",
      "UserId": "AIDA5WPGPNRISWOC332WS",
      "Arn": "arn:aws:iam:941618064465:user/Admin",
      "CreateDate": "2026-01-17T18:02:08+00:00",
      "PasswordLastUsed": "2026-01-18T04:38:23+00:00"
    }
  ]
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-groups
{
  "Groups": []
}
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$ aws iam list-access-keys --user-name MyUserCli
An error occurred (NoSuchEntity) when calling the ListAccessKeys operation: The user with name MyUserCli cannot be found.
codespaces-3c16f6:/workspaces/CC-HajraSarwar-022$
```

The screenshot displays the AWS Billing and Cost Management console. The left sidebar shows navigation options: Home, Getting Started, Dashboards, Billing and Payments (selected), and Cost and Usage Analysis. The main content area is titled 'Bills' and shows the 'AWS estimated bill summary' for the billing period of January 1 to January 31, 2026. The summary indicates a total in USD of USD 0.00, with the service provider being Amazon Web Services, Inc. The bill status is 'Pending'. The page refresh time is Sunday, January 18, 2026 at 12:05:14 PM GMT+5. The bottom of the console shows the footer with '© 2026, Amazon Web Services, Inc. or its affiliates.' and links for Privacy, Terms, and Cookie preferences.

Billing and Cost Management

Bills Info

Download all to CSV Print Billing period: January 2026

Page refresh time: Sunday, January 18, 2026 at 12:05:14 PM GMT+5

AWS estimated bill summary Info

Total charges and payment information

Account ID	Billing period Info	Bill status Info
941618064465	January 1 - January 31, 2026	Pending
Service provider	Total in USD	
Amazon Web Services, Inc.	USD 0.00	
Estimated grand total:		USD 0.00

► Payment information Info

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences